PRODUCT REQUIREMENTS DOCUMENT

Gamified eLearning System

A Dual-Adaptive, Glass-Box AI Architecture for Personalized Education

Document Version	1.0 — Pilot / 400-Level Project Release
Project Title	Gamified eLearning System (GELS)
Document Type	Product Requirements Document (PRD)
Prepared For	Junior Developer — Software Engineering Project
Status	Ready for Implementation
Tech Stack	Next.js · FastAPI · PostgreSQL · Reinforcement Learning
Date	2026

1. Product Vision

1.1 Problem Statement

Modern e-learning platforms function as rigid digital warehouses — they democratize access to information but fail to address the 'learning' side of the equation. Students log in but tune out. Two solutions have emerged in isolation:

- Adaptive Learning systems that optimize the curriculum (cognitive axis).
- Gamification systems that motivate students (affective axis).

The critical problem is that these two solutions are almost always developed in isolation. An adaptive system without motivational support feels robotic and dry. Gamification applied to a rigid curriculum is superficial 'sugar coating.' The result is a fragmented ecosystem that fails to sustain genuine, long-term learner engagement, contributing to the high dropout rates endemic to online education.

1.2 Target Users

- Primary: Undergraduate Software Engineering students (400-level)
- Secondary: Lecturers / Instructors who design and monitor courses
- Tertiary: System Administrators who maintain platform health and access control

1.3 Value Proposition

Core Promise
GELS is the first natively unified architecture where the Cognitive Engine (adaptive learning) and the Affective Engine (gamification) share a single data mining infrastructure — ensuring that content difficulty and motivational mechanics are always synchronized in real time.

- For students: Content that is always calibrated to their knowledge level, with rewards that match their psychological profile — eliminating both frustration and boredom.
- For instructors: Full transparency into AI decisions through Glass Box dashboards with manual override capabilities — keeping the human educator in the pedagogical loop.
- For institutions: Measurable reduction in dropout rates through a system grounded in Self-Determination Theory, Cognitive Load Theory, and Constructivist Learning principles.

1.4 Differentiation from Traditional LMS

Dimension	Traditional LMS	GELS
Content Delivery	Static, one-size-fits-all	Real-time RL-based sequencing
Gamification	None or uniform PBL	Adaptive per player-type profile

Dimension	Traditional LMS	GELS
AI Transparency	Black box or none	Glass Box — full instructor visibility
Data Architecture	Siloed modules	Unified EDM feeding both engines
Learner Modeling	Basic completion tracking	Continuous cognitive + affective profiling
Motivational Design	None	SDT-aligned, player-type mapped mechanics

2. User Personas

2.1 Persona A — The Learner (Software Engineering Student)

Name / Role	Chidera, 400-Level Software Engineering Student
Age	21–23 years old
Goals	Achieve mastery in core SE modules (algorithms, systems design, databases) at his own pace; earn visible proof of competence; stay motivated through a full semester.
Pain Points	Rigid LMS forces him through content he already knows; has no visibility into his progress trajectory; one-size-fits-all gamification (e.g. leaderboards) demotivates him when he falls behind; feels isolated learning alone online.
Behavioral Traits	Checks his phone frequently; responds strongly to short-term win feedback; prefers visual progress indicators; abandons tasks without immediate feedback; motivated by achievement badges rather than social comparison.
SDT Need Profile	Competence (primary) > Autonomy (secondary) > Relatedness (tertiary)

2.2 Persona B — The Instructor (Lecturer)

Name / Role	Dr. Adaeze, Senior Lecturer — Software Engineering Dept.
Age	38–50
Goals	Monitor student progress at both individual and cohort level; identify at-risk students early; understand WHY the AI makes specific content recommendations; override automated decisions when pedagogically necessary.
Pain Points	AI-driven systems are black boxes she cannot interpret; dashboards show raw data she cannot act on; no mechanism to intervene in the adaptive path manually; no transparency into why specific rewards were assigned.
Behavioral Traits	Prefers structured, actionable analytics over raw data; checks dashboards weekly; wants to upload new content and have the system adapt without re-engineering; values human judgment above algorithmic decisions.
Key Need	Glass Box AI — clear, human-readable explanations of every adaptive decision.

2.3 Persona C — The System Administrator

Name / Role	Emeka, IT/System Administrator
Age	25–35
Goals	Maintain platform uptime, manage user roles, audit security logs, ensure data integrity of learner profiles and EDM pipelines.

Pain Points	Complex microservices architectures are hard to debug; poor logging makes incident tracing difficult; no simple role-assignment interface.
Behavioral Traits	CLI-comfortable; prefers automation over manual processes; monitors system metrics daily; values clear API documentation.
Key Need	Role-based access control, clear audit logs, stable deployment infrastructure.

3. User Stories

3.1 Learner Stories

1. US-001: As a **learner**, I want *register and log in with a secure, role-assigned account*, so that I can access my personalized learning environment without risking data exposure.
2. US-002: As a **learner**, I want *complete a baseline diagnostic assessment on first login*, so that the system can build my initial cognitive profile and avoid recommending content I already know.
3. US-003: As a **learner**, I want *receive the next learning module automatically selected based on my performance*, so that I am always working at the right difficulty level — never bored, never overwhelmed.
4. US-004: As a **learner**, I want *earn points and badges that match my motivational style (not a one-size-fits-all set)*, so that my rewards feel meaningful and genuinely motivating rather than arbitrary.
5. US-005: As a **learner**, I want *view a personal progress dashboard showing my mastery level, XP, and badges*, so that I have a clear, visual sense of my academic trajectory and can self-regulate my learning.
6. US-006: As a **learner**, I want *access private quests and challenges rather than only public leaderboards*, so that I can compete against my own previous performance without the stress of public social ranking.
7. US-007: As a **learner**, I want *collaborate in team-based quests with classmates*, so that I can satisfy my need for relatedness and learn through peer interaction.
8. US-008: As a **learner**, I want *receive an immediate explanation when the system changes my learning path*, so that I understand why I am being redirected and do not feel the system is making arbitrary decisions.

3.2 Instructor Stories

9. US-009: As a **instructor**, I want *upload structured course modules that the adaptive engine can sequence*, so that I can control the curriculum content while allowing the AI to personalize delivery.
10. US-010: As a **instructor**, I want *view a class-level analytics dashboard showing engagement rates, drop-off points, and average mastery scores*, so that I can identify struggling cohorts and intervene before dropout occurs.
11. US-011: As a **instructor**, I want *see a human-readable explanation for every AI content recommendation made to any student*, so that I retain pedagogical authority and can validate that the AI's decisions are educationally sound.
12. US-012: As a **instructor**, I want *manually override the system's adaptive path for a specific student*, so that I can apply human judgment in cases where the algorithm may not account for contextual factors (e.g., a student's personal circumstances).
13. US-013: As a **instructor**, I want *configure which gamification mechanics are active for a given course (e.g., disable leaderboards for high-anxiety cohorts)*, so that I can tailor the motivational layer to the psychological needs of a specific group of students.

3.3 System / Admin Stories

14. US-014: As a **system administrator**, I want *assign and revoke user roles (learner / instructor / admin) from a management interface*, so that access control remains clean, secure, and auditable.
15. US-015: As a **system administrator**, I want *view system health logs and performance metrics for the AI engine and database*, so that I can proactively identify and resolve bottlenecks before they impact learner experience.

4. Functional Requirements

Architecture Mandate
The Cognitive Engine (learning adaptation) and the Affective Engine (gamification) are NOT separate modules. They share a single centralized Educational Data Mining (EDM) repository and communicate bidirectionally in real time.

4.1 AI-Powered Cognitive Engine (Google Gemini)

AI Model: Google Gemini 1.5 Flash (Free Tier)
The system leverages Google Gemini 1.5 Flash via the Google AI Studio API — available completely free with no credit card required (generous free-tier quota). Gemini replaces a custom-built RL model for the pilot, acting as the reasoning engine that evaluates the learner state vector and decides the next pedagogical action. This significantly reduces development complexity while delivering intelligent, explainable decisions out of the box. The API is accessed via the google-generativeai Python SDK (pip install google-generativeai). In Phase 2, the system can optionally add a custom fine-tuned layer on top of Gemini decisions.

4.1.1 State Representation

The system builds a structured learner state object at every decision point. This state is serialized as a JSON payload and injected into the Gemini prompt as context. The state includes:

State Variable	Data Type	Source / Notes
knowledge_level	Float [0.0–1.0] per topic	Derived from assessment scores via Bayesian Knowledge Tracing
engagement_score	Float [0.0–1.0]	Computed from time-on-task, interaction rate, drop-off events
current_difficulty	Enum: EASY / MEDIUM / HARD	Last module difficulty delivered
session_duration_mins	Integer	Current session length in minutes
consecutive_errors	Integer	Count of failed assessments in current session
player_type	Enum: ACHIEVER / SOCIALIZER / EXPLORER	Assigned during onboarding; updated via behavior
affective_state	Enum: FOCUSED / BORED / FRUSTRATED	Inferred from telemetry patterns
learning_style_pref	Enum: VISUAL / TEXTUAL / INTERACTIVE	Inferred from navigation behavior

4.1.2 Actions

Gemini is prompted with the learner state vector and a strict system prompt that constrains its response to one of six defined actions. The prompt format is: 'Given this learner state: {state_json}, choose the single best pedagogical action from: [ADVANCE, ESCALATE, REMEDIATE, SIMPLIFY,

TRIGGER_REWARD, PAUSE_AND_REFLECT]. Respond ONLY with the action name and a one-sentence rationale.' The available actions are:

- ADVANCE — Serve the next module at current difficulty.
- ESCALATE — Increase module difficulty by one level.
- REMEDIATE — Insert a review module targeting the identified knowledge gap.
- SIMPLIFY — Reduce difficulty; introduce an alternative explanation format.
- TRIGGER_REWARD — Signal the Affective Engine to deploy a motivational mechanic.
- PAUSE_AND_REFLECT — Present a metacognitive checkpoint prompt.

4.1.3 Scoring Signals (Gemini Decision Context)

These signals are computed by the EDM pipeline and passed directly into the Gemini prompt as part of the learner state. They inform Gemini's reasoning without requiring a custom reward function:

Event	Reward Value	Rationale
Correct answer, first attempt	+1.0	Confirms optimal difficulty calibration
Correct answer, 2nd–3rd attempt	+0.5	Partial mastery — system challenged appropriately
Assessment failure > 3 attempts	-0.8	Signal to SIMPLIFY or REMEDIATE
Session abandonment (drop-off)	-1.5	High-penalty signal; should be avoided
Engagement score increase	+0.3	Indirect reward for affective alignment
Completion with < avg. time	+0.2	Pace efficiency bonus

4.1.4 Real-Time Curriculum Sequencing

At the completion of every learning module or assessment, the system executes the following pipeline via FastAPI BackgroundTasks:

16. EDM pipeline aggregates latest telemetry from the event_log table and computes the current learner state object.
17. FastAPI serializes the state object to JSON and constructs the Gemini system prompt (see 4.1.2).
18. The google-generativeai SDK sends the prompt to Gemini 1.5 Flash API (async HTTP call, free tier).
19. Gemini returns: action name + one-sentence rationale string.
20. FastAPI parses the response, fetches the corresponding content module, and prepares the directive payload.
21. If action == TRIGGER_REWARD, a signal is passed to the Affective Engine for mechanic deployment.
22. The action, rationale string, and new state are written to the decision_log table (Glass Box record).
23. The directive payload is returned to the Next.js frontend as a JSON response.

4.2 Tailored Gamification System (Affective Engine)

4.2.1 Player-Type Mapping

The system classifies each learner into a dominant player type using behavioral data collected from the onboarding assessment and early session telemetry. Player types are based on the Bartle taxonomy adapted for educational contexts:

Player Type	Behavioral Signals	Primary Mechanics Served
ACHIEVER	High quiz-attempt rate; re-tries failed modules; focuses on badges	Badges, XP levels, personal bests, mastery streaks
SOCIALIZER	Opens leaderboard; participates in discussion forums; forms study groups	Leaderboards, team quests, peer-recognition badges, collaboration rewards
EXPLORER	Accesses optional content; clicks supplementary links; navigates non-linearly	Unlock hidden content, narrative quests, exploration bonus XP, knowledge maps

Player type is not static. The system recalculates the dominant player type every 5 sessions based on updated behavioral telemetry and blends mechanics accordingly.

4.2.2 Gamification Mechanics Specification

- **Awarded on every interaction. Base XP = difficulty multiplier × speed bonus. XP feeds into a visible level-up system with meaningful curriculum-aligned tier names (e.g., Junior Dev → Mid Dev → Senior Dev).:** Points / XP
- **28 predefined achievement badges mapped to specific learning milestones. Badges are visually distinct for ACHIEVER, SOCIALIZER, and EXPLORER categories. Badges surface on the learner dashboard and are shareable externally.:** Badges
- **Two types: (1) Global class leaderboard (visible to all); (2) Private leaderboard showing only the top 10% of peers near the student's level. The system suppresses the global leaderboard for learners classified as anxious (detected via consecutive failure patterns).:** Leaderboards
- **Solo challenge missions unlocked adaptively based on the learner's current knowledge level. Quests carry narrative framing to satisfy the EXPLORER type (e.g., 'Debug the Production Incident').:** Private Quests
- **Group quests requiring 3–5 participants to collaboratively solve a multi-module challenge. A shared team badge is issued upon completion. Particularly triggered for SOCIALIZER profiles.:** Team / Collaboration Features
- **Daily and weekly login streaks tracked and displayed. Streak bonuses (+15% XP) applied as a retention mechanic for learners with high historical drop-off probability.:** Streaks

4.2.3 Dynamic Mechanic Assignment

The Affective Engine uses the following logic to assign mechanics at runtime:

24. On TRIGGER_REWARD signal from the RL engine, the Affective Engine reads the learner's current player_type, engagement_score, and session_duration.
25. If engagement_score < 0.4 AND player_type == ACHIEVER → Deploy a personal best challenge notification.
26. If engagement_score < 0.4 AND player_type == SOCIALIZER → Trigger a peer collaboration invite notification.

27. If consecutive_errors >= 3 → Suppress competitive mechanics (leaderboard, peer comparison); deploy an encouraging private quest instead.
28. If affective_state == BORED AND knowledge_level > 0.8 → Unlock an EXPLORER hidden content challenge.
29. Log the mechanic deployment event to the EDM for both the Instructor Dashboard and future model training.

4.3 Educational Data Mining (EDM)

4.3.1 Telemetry Data Collected

Telemetry Signal	Collection Method	Feeds Engine
Time-on-task per module	Client-side JS timer → API payload	Both (pacing signal for RL; boredom signal for Affective)
Navigation behavior (page jumps, backtracking)	Frontend event tracking → log stream	Cognitive (confusion detection); Affective (EXPLORER profiling)
Quiz / assessment scores	Backend scoring pipeline	Cognitive (BKT state update); Affective (reward trigger)
Drop-off points (session abandonment)	Session timeout event	Cognitive (negative reward); Affective (streak break alert)
Interaction rate (clicks / min)	Frontend event throttling	Affective (engagement_score computation)
Assessment attempt count	Backend counter	Cognitive (difficulty signal); Affective (frustration detection)
Peer interaction frequency	Social feature event log	Affective (SOCIALIZER profiling)
Optional content access	Content click events	Affective (EXPLORER profiling)

4.3.2 EDM Processing Pipeline

All telemetry events are written directly to the event_log table in PostgreSQL in real time. After each assessment submission or module completion, the FastAPI endpoint enqueues an EDM computation job using FastAPI's built-in BackgroundTasks — no extra dependencies (no Celery, no Redis). The pipeline runs asynchronously after the HTTP response is returned to the client:

30. FastAPI BackgroundTask reads all unprocessed event_log rows for the learner (rows where processed = FALSE).
31. Aggregate raw events into derived metrics: engagement_score, consecutive_errors, affective_state.
32. Update the learner_profile record in PostgreSQL with the latest state vector.
33. Construct Gemini prompt from updated state; call Gemini 1.5 Flash API; parse action + rationale.
34. If reward conditions are satisfied, execute Affective Engine logic and update gamification tables.
35. Write the action + rationale to the decision_log table (supports Glass Box Instructor Dashboard).
36. Mark processed event_log rows as processed = TRUE.

Database-Only Architecture Note

All telemetry, events, and logs are stored exclusively in PostgreSQL for the pilot. The event_log table is designed with a flexible JSONB payload column so that, in Phase 2, the developer can optionally introduce MongoDB for the event log by implementing an alternative EventLogRepository class — swapping the storage backend without changing any business logic. See Section 5.4 for the repository pattern specification.

5. Technical Architecture

5.1 System Overview

The system is organized as a decoupled, three-tier architecture with a shared EDM data layer:

```
HIGH-LEVEL ARCHITECTURE

┌──────────────────────────────────────────────────────────────────────────┐
│                    PRESENTATION LAYER (Next.js)                          │
│  Learner Dashboard | Instructor Dashboard | Admin Panel                  │
└──────────────────────────────────────────────────────────────────────────┘
                          │ REST API (JSON)
                          ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                     LOGIC LAYER (FastAPI)                                │
│  ┌──────────────────────────┐   ┌──────────────────────────────┐         │
│  │  COGNITIVE ENGINE        │◄──►│  AFFECTIVE ENGINE (Gamif.)   │         │
│  │  Gemini 1.5 Flash API    │   │  Player-Type | Mechanics     │         │
│  │  State→Prompt→Action     │   │  SDT-aligned rewards         │         │
│  └──────────────────────────┘   └──────────────────────────────┘         │
│   FastAPI BackgroundTasks (built-in) — no Celery, no Redis               │
│                  └──────────────┬──────────────┘                          │
│                  ┌──────────────▼──────────────┐                          │
│                  │  EDM Pipeline (Shared Data Layer) │                    │
└──────────────────────────────────────────────────────────────────────────┘
                          │ SQLAlchemy ORM (PostgreSQL only)
                          ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                DATA PERSISTENCE LAYER (PostgreSQL only)                  │
│  users | learner_profiles | courses | modules | event_log (JSONB)        │
│  gamification_rewards | decision_log | badges | quests                   │
│                                                                          │
│  Repository Pattern → swap event_log to MongoDB in Phase 2               │
│  with zero changes to business logic                                     │
└──────────────────────────────────────────────────────────────────────────┘
```

Data Flow Summary:

- Learner interaction events flow from the Next.js client to the FastAPI backend via REST API calls.
- FastAPI processes events synchronously (returns HTTP response immediately), then enqueues an EDM computation job via BackgroundTasks.
- The BackgroundTask reads unprocessed event_log rows, computes the updated learner state, and calls the Gemini 1.5 Flash API with a structured prompt.
- Gemini returns a next action + rationale. FastAPI writes the decision to decision_log and updates learner_profile.
- On the next frontend request (e.g. loading the next module), the pre-computed directive is already waiting in the DB — no blocking AI call on the critical path.
- The Affective Engine evaluates gamification conditions within the same BackgroundTask and updates reward tables accordingly.

5.2 Tech Stack

Layer	Technology	Justification
Frontend	Next.js 14+ (React)	SSR, fast hydration, component-driven state management for dynamic gamification UI
Backend	FastAPI (Python)	Async request handling; native Python ecosystem; built-in BackgroundTasks — no extra broker needed
AI / Cognitive Engine	Google Gemini 1.5 Flash (google-generativeai SDK)	Free tier via Google AI Studio; no payment required; strong reasoning for structured prompt-action tasks
Background Tasks	FastAPI BackgroundTasks (built-in)	Zero extra dependencies; runs EDM pipeline after HTTP response — no Celery, no Redis, no broker
Database	PostgreSQL 15+ (only)	Single DB for pilot; JSONB column on event_log handles flexible telemetry; ACID compliant
ORM / Migrations	SQLAlchemy + Alembic	Clean model definitions; schema versioning; repository pattern enables future NoSQL swap
Authentication	JWT (via python-jose)	Stateless, role-aware token auth across frontend and API
Deployment (pilot)	Docker Compose	Single docker-compose.yml spins up Next.js + FastAPI + PostgreSQL on a developer laptop
Future NoSQL (Phase 2)	MongoDB (optional swap)	EventLogRepository interface allows MongoDB to replace event_log table with zero business logic changes

5.3 API Design

5.3.1 Authentication Endpoints

Method	Endpoint	Description	Request Body	Response
POST	`/api/auth/register`	Create a new user account	`{"email":"","password":"","role":"learner"}`	`{"user_id":"uuid","token":"jwt"}`

Method	Endpoint	Description	Request Body	Response
POST	`/api/auth/login`	Authenticate and	`{"email":"","password":""}`	`{"token":"jwt","expires_in":3600}`

Method	Endpoint	Description	Request Body	Response
		receive JWT		

5.3.2 Learning Endpoints

Method	Endpoint	Description	Request Body	Response
GET	/api/learning/next-module	Get next adaptive content for learner	Headers: Authorization: Bearer <jwt>	{"module_id":"","title":"","difficulty":"MEDIUM","content_url":"","rationale":"..."}

Method	Endpoint	Description	Request Body	Response
POST	/api/learning/submit-assessment	Submit quiz answers; triggers EDM pipeline	{"module_id":"","answers":[...]}	{"score":0.85,"passed":true,"next_action":"ADVANCE"}

5.3.3 Gamification Endpoints

Method	Endpoint	Description	Request Body	Response
GET	/api/gamification/profile	Fetch learner XP, badges, level, streaks	Headers: Authorization: Bearer <jwt>	{"xp":2400,"level":3,"badges":[...],"streak_days":5}

Method	Endpoint	Description	Request Body	Response
GET	/api/gamification/leaderboard	Get contextual leaderboard (filtered by	Query: ?scope=private	{"entries":[{"rank":1,"username":"","xp":2800},...]}

Method	Endpoint	Description	Request Body	Response
		player type)		

5.3.4 Instructor / Analytics Endpoints

Method	Endpoint	Description	Request Body	Response
GET	/api/instructor/cohort-analytics	Get aggregated cohort engagement metrics	Query: ?course id =<id>	{"avg_mastery":0.72,"drop_off_rate":0.18, "at_risk_count":4}

Method	Endpoint	Description	Request Body	Response
POST	/api/instructor/override-path	Manually set next module for a student	{"student_id":"","module_id":"","reason":"..."}	{"success":true,"override_logged":true}

Method	Endpoint	Description	Request Body	Response
GET	/api/instructor/decision-log	Retrieve Glass Box AI decision log for a student	Query: ?student_id=<id>&limit=20	{"decisions":[{"timestamp":"","action":"REMEDIATE","rationale":"Student scored below 0.4 on loops topic"},...]}

5.4 Data Models

5.4.1 User

Field	Type / Constraint	Description
user_id	UUID, PK	Unique identifier

Field	Type / Constraint	Description
email	VARCHAR(255), UNIQUE	Login credential
password_hash	VARCHAR(255)	bcrypt-hashed password
role	ENUM(learner, instructor, admin)	Drives RBAC logic
created_at	TIMESTAMP	Account creation time
last_active	TIMESTAMP	Updated on every API call

5.4.2 Learner Profile (Cognitive + Affective State)

Field	Type / Constraint	Description
profile_id	UUID, PK	Unique profile identifier
user_id	UUID, FK → users	One-to-one with user
knowledge_state	JSONB	Map of {topic_id: mastery_score} from BKT
player_type	ENUM	ACHIEVER / SOCIALIZER / EXPLORER
engagement_score	FLOAT(3)	Rolling 5-session average
xp_total	INTEGER	Cumulative experience points
current_level	INTEGER	Derived from xp_total thresholds
streak_days	INTEGER	Consecutive login days
last_updated	TIMESTAMP	Updated by EDM pipeline

5.4.3 Course & Learning Module

Field	Type / Constraint	Description
module_id	UUID, PK	Unique module identifier
course_id	UUID, FK → courses	Parent course
title	VARCHAR(255)	Display name
content_url	TEXT	CDN link to content asset
difficulty	ENUM(EASY,MEDIUM,HARD)	Used by RL state representation
topic_id	UUID, FK → topics	Enables BKT topic-level tracking
prerequisites	UUID[]	Array of module_ids that must be completed first
estimated_minutes	INTEGER	Used in time-on-task calculations

5.4.4 Learning Session & Event Log (PostgreSQL)

All telemetry is stored in a single event_log table in PostgreSQL. The JSONB payload column provides the flexibility of a document store within a relational database — meaning the developer does not need MongoDB for the pilot. Each event type stores a different payload shape inside the same column.

Field	Type / Constraint	Description
event_id	UUID, PK	Unique event identifier
user_id	UUID, FK → users	Owning learner
session_id	UUID	Groups events into a single session
event_type	ENUM	MODULE_START / ASSESSMENT_SUBMIT / DROP_OFF / REWARD_EARNED / NAV_JUMP / etc.
module_id	UUID, FK → modules (nullable)	Relevant module, if applicable
payload	JSONB	Flexible event data — e.g. {score:0.8, time_spent:420} or {from_page:3, to_page:7}
processed	BOOLEAN, DEFAULT FALSE	Set to TRUE after BackgroundTask has consumed this row
timestamp	TIMESTAMP WITH TIME ZONE	Precise event time; indexed for EDM queries

5.4.5 Repository Pattern — Future NoSQL Flexibility

Design Decision: Repository Pattern
The event_log data access is wrapped in an EventLogRepository interface with two concrete implementations: PostgreSQLEventLogRepository (active in Phase 1) and MongoEventLogRepository (available in Phase 2). All business logic in the EDM pipeline calls the interface — never the concrete class directly. To switch to MongoDB, the developer only changes a single dependency injection line in main.py. No service code changes required.

Pseudo-code interface (Python):

```python
# repositories/event_log_repo.py
class EventLogRepository(ABC):
    def write_event(self, event: EventLog) -> None: ...
    def get_unprocessed(self, user_id: str) -> list[EventLog]: ...
    def mark_processed(self, event_ids: list[str]) -> None: ...

# Phase 1 — PostgreSQL active
class PostgreSQLEventLogRepository(EventLogRepository): ...

# Phase 2 — swap in by changing ONE line in main.py
class MongoEventLogRepository(EventLogRepository): ...
```

5.4.5 Reward / Gamification Event Log

Field	Type / Constraint	Description
reward_id	UUID, PK	Unique reward event
user_id	UUID, FK → users	Recipient learner
reward_type	ENUM	XP_GRANT / BADGE_AWARD / LEVEL_UP / STREAK_BONUS
reward_value	JSONB	e.g., {"xp": 50} or {"badge_id": "uuid"}

Field	Type / Constraint	Description
trigger_reason	TEXT	Human-readable rationale (feeds Glass Box dashboard)
triggered_by	ENUM(SYSTEM, INSTRUCTOR)	Differentiates AI vs. manual rewards
timestamp	TIMESTAMP WITH TIME ZONE	Event time

6. Non-Functional Requirements

Quality Attribute	Requirement	Implementation Guidance
Usability	UI must be learnable within 5 min for new users; zero unexplained UI state changes	Minimalist Next.js UI; gamification overlays as non-blocking notifications; mobile-responsive
Performance	RL next-action computation < 300ms P95; reward delivery < 200ms P95	Async FastAPI handlers; pre-computed Q-tables cached in Redis for pilot
Scalability	Support 500+ concurrent learners without API latency degradation	Horizontal FastAPI scaling via Docker; PostgreSQL connection pooling (pgBouncer)
Reliability	99.5% uptime; zero data loss on session interruption	PostgreSQL WAL replication; session state persisted server-side; retry logic on client
Security	All passwords bcrypt-hashed; all API routes JWT-protected; all DB connections TLS-encrypted	python-jose for JWT; SQLAlchemy parameterized queries; HTTPS enforced in production
Transparency (Glass Box)	Every AI decision must have a logged, human-readable rationale accessible to instructors	decision_log table mandatory; rationale string required on all RL action commits
Privacy / Data Ethics	Learner telemetry data not shared externally; users can request data deletion	Data minimization in schema; soft-delete patterns; GDPR-aligned consent flow

7. Instructor Dashboard — Glass Box AI

The Instructor Dashboard is not an analytics add-on. It is a core architectural component that ensures the human educator retains pedagogical authority over every AI decision. The Glass Box principle mandates that no algorithmic action should be irreversible or opaque to an instructor.

7.1 Dashboard Panels & Metrics

Panel	Metrics Displayed	Visualization Type
Cohort Overview	Avg. mastery per topic; engagement distribution; at-risk count	Heat map + summary cards
Individual Student View	Knowledge state per topic; engagement timeline; current RL state; recent rewards	Line chart + state vector card
AI Decision Log	Last 20 RL actions with rationale text; override history	Scrollable table with filter
Drop-Off Analysis	Session abandonment events mapped by module; time-of-day heatmap	Bar chart + module risk score
Gamification Audit	Badges and XP granted per student; mechanic type distribution across cohort	Pie chart + per-student log
Mechanic Configuration	Toggle leaderboard visibility; adjust XP thresholds; enable/disable player-type detection	Form controls with live preview

7.2 Glass Box Examples

Example A — Content Recommendation Explanation
Why was this content recommended to Chidera? System Decision: Module 14 (Recursion — EASY difficulty) was recommended. Rationale: Chidera's knowledge_state for 'recursion' = 0.28 (below mastery threshold 0.70). He scored 40% on the previous assessment (Module 12) with 4 attempts. Engagement score = 0.35 (below average). Action: REMEDIATE. The system inserted a simpler foundational module before attempting Module 13 again. Instructor Actions Available: [Override to Module 13] [Mark as Understood — Skip Remediation] [Flag for Manual Review]

Example B — Reward Decision Explanation
Why did Chidera receive the 'Persistence Champion' badge? System Decision: Badge awarded after Session 18. Rationale: Chidera completed 5 consecutive sessions with at least one failed assessment followed by a retry-and-pass outcome. player_type = ACHIEVER. Trigger condition: consecutive_retry_success >= 5. Badge type: ACHIEVER category — personal mastery signal. Instructor Actions Available: [Revoke Badge] [Add Manual Bonus XP] [View Full Session History]

7.3 Override Capabilities

- Override next content module for any student (with mandatory reason field, logged permanently).
- Manually award or revoke XP and badges (logged with instructor ID and timestamp).
- Disable specific gamification mechanics for the entire cohort or individual students.
- Pause AI adaptation for a student and set manual content path for a defined period.
- Inject a custom message or resource link into a student's learning interface.

8. Cold Start Strategy

Critical Problem
A new learner has zero interaction history. The RL agent cannot compute a meaningful next action without a state vector. The system must collect enough information to bootstrap personalization on the first session — without being intrusive or overwhelming.

8.1 Baseline Assessment (Onboarding Quiz)

On first login, every new learner completes a mandatory 15-question adaptive baseline assessment before accessing any course content. The assessment covers core prerequisite topics for the enrolled course.

- Format: Multiple choice, timed (30 seconds per question).
- Adaptive: If the learner answers correctly 3 times in a row, question difficulty increases. If they fail twice, difficulty drops (mini-RL within the assessment itself).
- Duration: Approximately 8–12 minutes.
- Output: An initial knowledge_state vector written to the learner_profile table. Initial difficulty assignment (EASY, MEDIUM, or HARD starting point).

8.2 Player-Type Onboarding Survey

After the baseline assessment, a 5-question onboarding survey collects self-reported behavioral preferences to seed the player_type field:

37. 'When you learn something new, you prefer to: (a) master it completely, (b) discuss it with peers, (c) explore related topics.'
38. 'When you complete a challenge, what feels most rewarding: (a) seeing your score improve, (b) sharing the achievement, (c) unlocking new content.'

Survey responses map directly to player_type: A-answers → ACHIEVER, B-answers → SOCIALIZER, C-answers → EXPLORER. Ties default to ACHIEVER; type recalculates after 5 sessions.

8.3 Default Learner Profiles

While the baseline assessment and survey are being completed, the system uses a default profile template to ensure the RL agent has a non-null state to work with:

knowledge_level	0.5 (neutral starting point — no prior assumption)
engagement_score	0.6 (optimistic default — system assumes initial motivation)
difficulty	EASY (conservative start to build confidence)
player_type	ACHIEVER (most common type in SE student populations)
affective_state	FOCUSED

8.4 Progressive Personalization Schedule

Session	System Behavior	Personalization Depth
1 (Onboarding)	Baseline assessment + survey; static default profile active	0% — default rules only
2–3	EDM collects first telemetry; minor difficulty adjustment begins	25% — knowledge_state partially populated
4–5	Player-type refined from behavioral data; gamification mechanics begin adapting	50% — engagement_score and player_type reliable
6–10	RL agent has sufficient (S, A, R, S') tuples for confident path decisions	75% — full RL sequencing active
11+	Full dual-adaptive operation; affective_state inference active	100% — complete personalization

9. Implementation Roadmap

Phase 1 — MVP (Pilot Project Scope)

Goal
Build a functional, deployable prototype that demonstrates the core dual-adaptive loop. Suitable for a 400-level Software Engineering project submission and initial user testing with a small cohort (20–50 students).

Sprint	Deliverable	Definition of Done
S1	Authentication + User Profiles	Register/login flow; JWT auth; role-based access; PostgreSQL schema deployed
S2	Course & Module Management	Instructor can upload modules; learner can navigate and view content; basic assessment submission
S3	EDM Pipeline (FastAPI BackgroundTasks)	Telemetry events logged to event_log (PostgreSQL, JSONB payload); processed flag working; BackgroundTask enqueued on assessment submit; learner_profile updated
S4	Gemini Cognitive Engine	Google AI Studio API key setup; google-generativeai installed; state→prompt→action pipeline working; decision_log table populated
S5	Gamification Engine (Static Mechanics)	XP system, badge awards, streaks; player-type survey; rewards logged to DB
S6	Learner Dashboard	Progress visualization, badge display, leaderboard (global only in pilot); Glass Box rationale text displayed
S7	Instructor Dashboard (Basic)	Cohort mastery table; individual student state view; AI decision log accessible; basic override
S8	Cold Start + Onboarding	Baseline assessment quiz; player-type survey; default profile bootstrapped
S9	Integration Testing + Bug Fix	Full user journey tested end-to-end; all critical paths passing
S10	Pilot Evaluation	Deploy to test cohort; collect engagement analytics; document results

Phase 2 — Enhanced Intelligence

Goal
Improve the quality of Gemini's decisions by adding few-shot learning examples from real pilot data. Add dynamic player-type recalculation, private leaderboards, team quests, and richer affective state inference.

- Build a few-shot prompt library: after the pilot, curate the best (state → action) decisions from decision_log and inject them as examples into the Gemini system prompt for higher accuracy.
- Optionally introduce Bayesian Knowledge Tracing (BKT) to compute topic-level knowledge_state more precisely, feeding richer context into the Gemini prompt.
- Add private leaderboard (top-10 peer-filtered) and team quest mechanics for SOCIALIZER profiles.
- Add affective_state inference from telemetry patterns (time-on-task anomalies, error spikes).
- Build the full Mechanic Configuration panel in the Instructor Dashboard.
- Optionally introduce MongoDB as an alternative EventLogRepository — swap in via one line in main.py (see Section 5.4.5) if the developer's environment supports it.

Phase 3 — Full Production System

Goal
Scale to multi-course, multi-institution deployment. Add explainable AI (XAI) narrative generation, collaborative social features, and longitudinal learning analytics.

- Multi-course support with prerequisite graphs and cross-course adaptive sequencing.
- Upgrade to Gemini's instruction-tuned API or a fine-tuned model trained on accumulated decision_log data from the pilot and Phase 2.
- Full collaborative features: real-time team quests, peer review system, group badge economy.
- Longitudinal analytics: track learner trajectories across semesters; exportable to institution SITS.
- Accessibility compliance: WCAG 2.1 AA; screen-reader support for all gamification UI.
- Mobile app (React Native) sharing the same FastAPI backend.
- A/B testing framework for gamification mechanic experiments across cohorts.

Appendix A — Theoretical Grounding & Implementation Notes

This PRD is grounded in three established theoretical frameworks drawn directly from the source draft, plus a practical note on the Gemini AI integration:

A.0 Gemini 1.5 Flash — Setup Notes for Developer

Getting Started (Free, No Credit Card)
1. Go to https://aistudio.google.com — sign in with any Google account. 2. Click 'Get API Key' → 'Create API Key in new project'. 3. Copy the key and add it to your FastAPI .env file as: GEMINI_API_KEY=your_key_here 4. Install the SDK: pip install google-generativeai 5. Free tier limits: 15 requests/min, 1 million tokens/day — more than sufficient for a pilot with 50 students. 6. Model to use: gemini-1.5-flash (fastest, most cost-efficient on the free tier).

A.1 Self-Determination Theory (SDT)

Developed by Deci and Ryan. The three psychological needs driving all gamification mechanic assignments in GELS:

- Competence: Satisfied by correctly calibrated difficulty (RL engine) and XP / badge feedback (Affective Engine). Never serve content that is so hard it causes repeated failure without scaffolding.
- Autonomy: Satisfied by choice-based learning paths, private quests (learner controls pace), and the ability to opt out of social leaderboards.
- Relatedness: Satisfied by team quests, social badges, and peer leaderboards (for SOCIALIZER types only).

A.2 Cognitive Load Theory

The RL engine's primary constraint: do not serve content that causes extraneous cognitive load. Operationalized as:

- If consecutive_errors >= 3 → the current difficulty is too high → action: SIMPLIFY or REMEDIATE.
- If time_on_task >> estimated_minutes → learner is struggling → reduce difficulty.
- If time_on_task << estimated_minutes AND score > 0.9 → learner is bored → action: ESCALATE.
- Affective Scaffolding: when cognitive load is high, the Affective Engine suppresses competitive mechanics and deploys supportive, low-pressure rewards to prevent learned helplessness.

A.3 Affective Scaffolding

Affective Scaffolding is the principle that the emotional/motivational environment must be actively managed alongside the cognitive environment — not left to chance. In GELS, this means:

- The Affective Engine does not wait for the learner to disengage before acting. It proactively deploys mechanics when engagement_score falls below defined thresholds.
- Mechanics are not chosen randomly. They are algorithmically matched to player_type and affective_state to maximize resonance with the learner's current psychological condition.
- The affective layer is bidirectional: it informs the RL engine (a highly engaged learner can tolerate higher difficulty increments) and is informed by it (a REMEDIATE action signals the Affective Engine to deploy an encouraging reward rather than a competitive one).